

Expense Tracker - Technical Specification & Architecture Document

Table des Matières

1. Executive Summary & Architecture Overview

- 1.1 Executive Brief
- 1.2 Maturity Assessment
- 1.3 Technical Stack
- 1.4 Architectural Constraints
- 1.5 Critical Dependencies

2. Architecture Workflows & Visual Diagrams

3. Detailed Technical Specifications & Business Rules

- 3.1 Requirements Traceability
- 3.2 Security Rules
- 3.3 Data Models

4. Project Governance & Structural Gaps

- 4.1 Structural Gaps
- 4.2 Remediation & Workflow

5. Technical & Domain Glossary (Terminology Reference)

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The Expense Tracker is a single-user financial management application built on the Next.js App Router. It utilizes a client-side data pattern leveraging localStorage for persistence, ensuring a lightweight, browser-based state management system. The architecture is designed for future-readiness by isolating shared types and validation utilities within a dedicated server-side directory structure.

1.2 Maturity Assessment

The project is currently in a REFINEMENT state. While the core framework and testing strategy are defined, there are critical structural gaps regarding the absence of formal Data Models and Schemas, as well as undefined API Contracts. The lack of a defined Expense entity represents a high-severity omission that must be resolved before full-scale implementation.

1.3 Technical Stack

- Next.js
- TypeScript
- Tailwind CSS
- Jest
- React Testing Library
- Node 18+

1.4 Architectural Constraints

- Persistence limited to browser-scope localStorage.
- Strict directory isolation for shared types and validation in `src/server`.
- Mandatory unit test implementation prior to merge as per repository guidelines.

1.5 Critical Dependencies

- localStorage browser API for data persistence.
- `src/server` directory for shared type definitions.
- Jest/RTL test suite for validator and component verification.
- `research.md` for recording technical decisions from Phase 0.

2. Architecture Workflows & Visual Diagrams

(No diagrams provided in source data)

3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

Identifier	Requirement / Choice	Source Section	Description
STACK-NEXTJS	Architecture Choice	Technical Context	Next.js App Router with TypeScript and Tailwind CSS
STORAGE-LOCAL	Architecture Choice	Technical Context	Client-side persistence using localStorage
TEST-STRAT	Architecture Choice	Technical Context	Unit tests using Jest and React Testing Library for validators and components
STRUC-SERVER-FOLDER	Architecture Choice	Project Structure	Use src/server for shared types and validation utilities to ensure future-readiness
PHASE-0-RESEARCH	Task	Complexity Tracking	Conduct research to resolve technical unknowns and record decisions in research.md
PHASE-2-TESTING	Task	Constitution Check	Implement unit tests before merge as per repository guidelines

3.2 Security Rules

(No specific security rules provided; see Section 4.1 for structural gaps regarding Security & Identity)

3.3 Data Models

(No data models provided; see Section 4.1 for structural gaps)

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Missing Section	Priority	Remediation Advice
Data Models & Schemas	HIGH	Define the Expense entity and its properties in a dedicated data-model.md file.
API Contracts & Flow	MEDIUM	Define the contracts for the future-ready route handlers in expenses-api.md.
Security & Identity	LOW	Document the explicit decision to omit authentication for this single-user local app.
Open Questions & Uncertainties	MEDIUM	List specific unknowns to be resolved during Phase 0 research.

4.2 Remediation & Workflow

The project follows a phased approach:

- 1. **Phase 0 (Research)**: Resolve technical unknowns and document in ``research.md`` (linked to ``PHASE-0-RESEARCH``).
- 2. **Implementation**: Build components and logic following the ``STRUC-SERVER-FOLDER`` isolation pattern.
- 3. **Phase 2 (Testing)**: Execute ``TEST-STRAT`` via ``PHASE-2-TESTING`` before any merge.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
CSS	TECHNICAL_STACK	STACK-NEXTJS	The styling layer implemented via a utility-first framework to manage visual presentation.
LocalStorage	TECHNICAL_STACK	STORAGE-LOCAL	The browser-based key-value mechanism used for persistent data retention without a backend.
TypeScript	TECHNICAL_STACK	STACK-NEXTJS	The strongly typed superset of JavaScript used to ensure type safety across the application and shared server utilities.